TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
HCM·UTE
HCMC University of Technology and Engineering

*ARIN330585 – Trí tuệ nhân tạo*

# Introduction to Artificial Intelligence

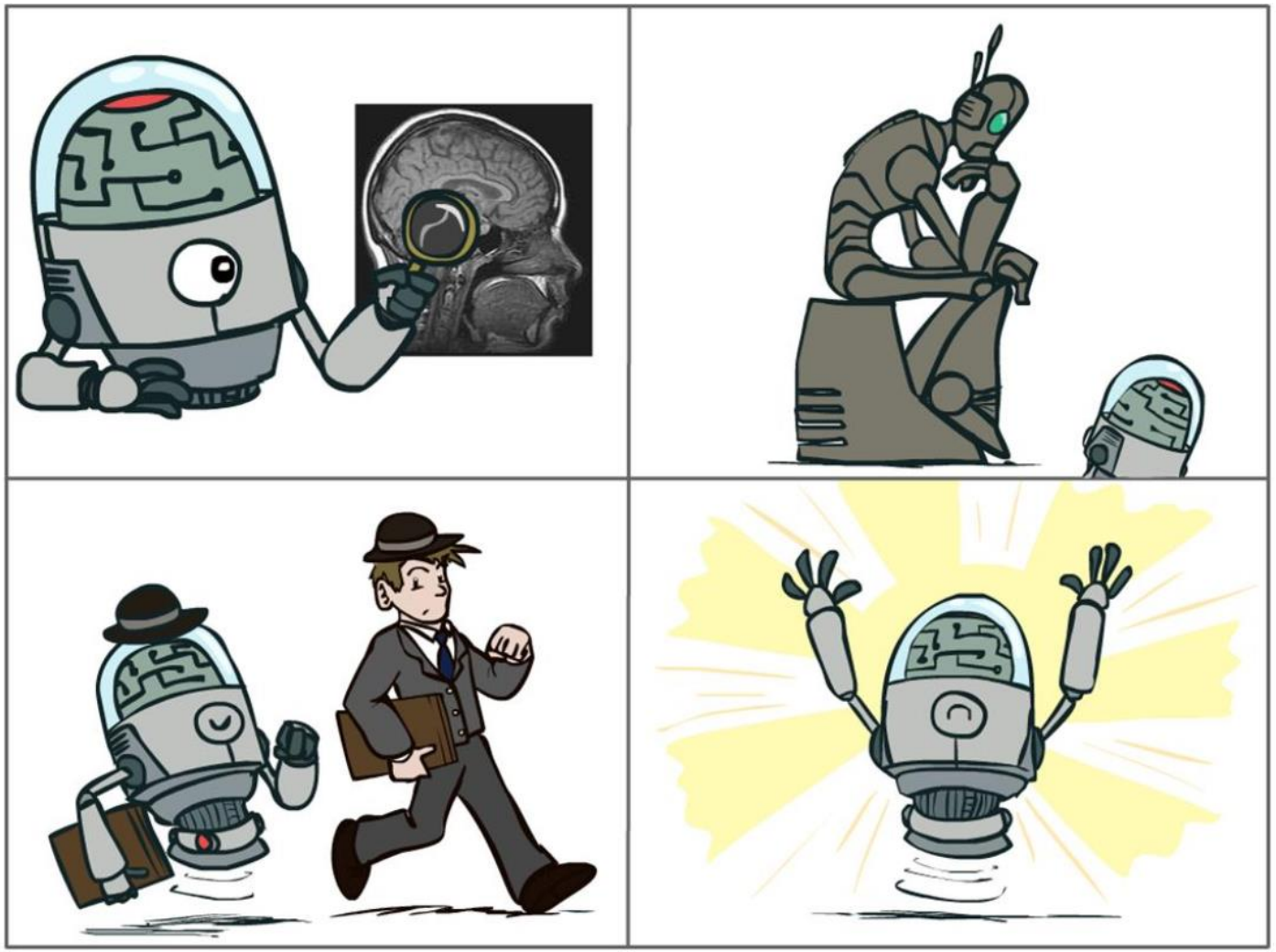
**NCS. Võ Long Tuấn**

# Chapter 4: Intelligent Systems

1. *Definition of AI*
2. *AI Agents*

# 1. What is AI?

Think like people



Think rationally

Act like people

Act rationally

# 1. What is AI?

| **Thinking Humanly** | **Thinking Rationally** |
|---|---|
| "The exciting new effort to make computers think . . . *machines with minds*, in the full and literal sense." (Haugeland, 1985)<br><br>"[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . ." (Bellman, 1978) | "The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985)<br><br>"The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992) |
| **Acting Humanly** | **Acting Rationally** |
| "The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990)<br><br>"The study of how to make computers do things at which, at the moment, people are better." (Rich and Knight, 1991) | "Computational Intelligence is the study of the design of intelligent agents." (Poole *et al.*, 1998)<br><br>"AI . . . is concerned with intelligent behavior in artifacts." (Nilsson, 1998) |

# 1. What is AI?

"Artificial intelligence (AI) refers to systems that display intelligent behaviour by analyzing their environment and taking actions – with some degree of autonomy – to achieve specific goals.

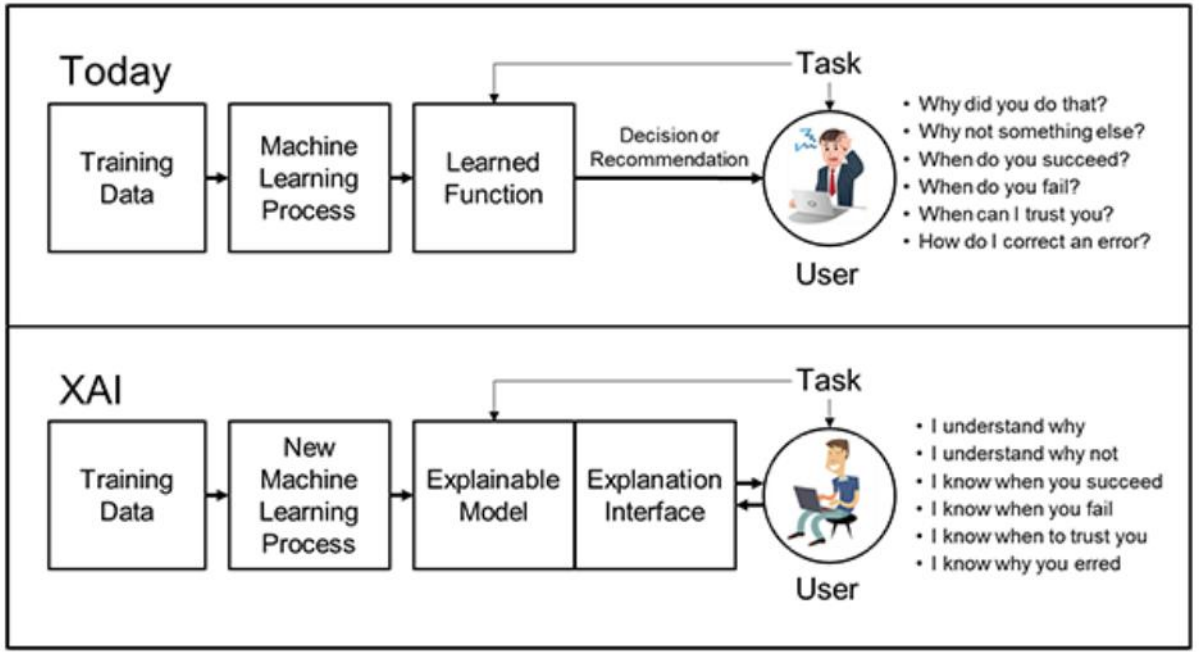
AI-based systems can be purely software-based, acting in the virtual world (*e.g. voice assistants, image analysis software, search engines, speech and face recognition systems*) or AI can be embedded in hardware devices (*e.g. advanced robots, autonomous cars, drones or Internet of Things applications*)."


European Commission: https://digital-strategy.ec.europa.eu,
Document made public on 8 April 2019.

E**x**plainable **AI** (XAI)

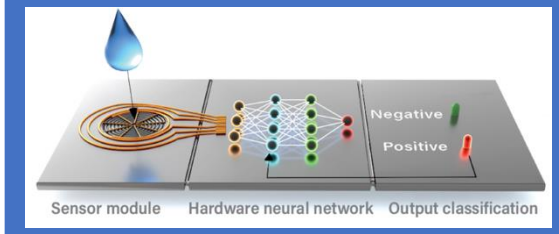
*"New machine-learning systems will have the ability to **explain their rationale**, characterize their strengths and weaknesses, and **convey an understanding of how they will behave in the future**. [...] These models will be combined with state-of-the-art human-computer interface techniques capable of translating models into understandable and useful **explanation dialogues for the end user**."*

Source: DARPA, www.darpa.mil



# Approaches to AI

Thinking humanly: The cognitive modeling approach




Thinking rationally: The laws of thought approach

Focusing on logic

4 main approaches to AI

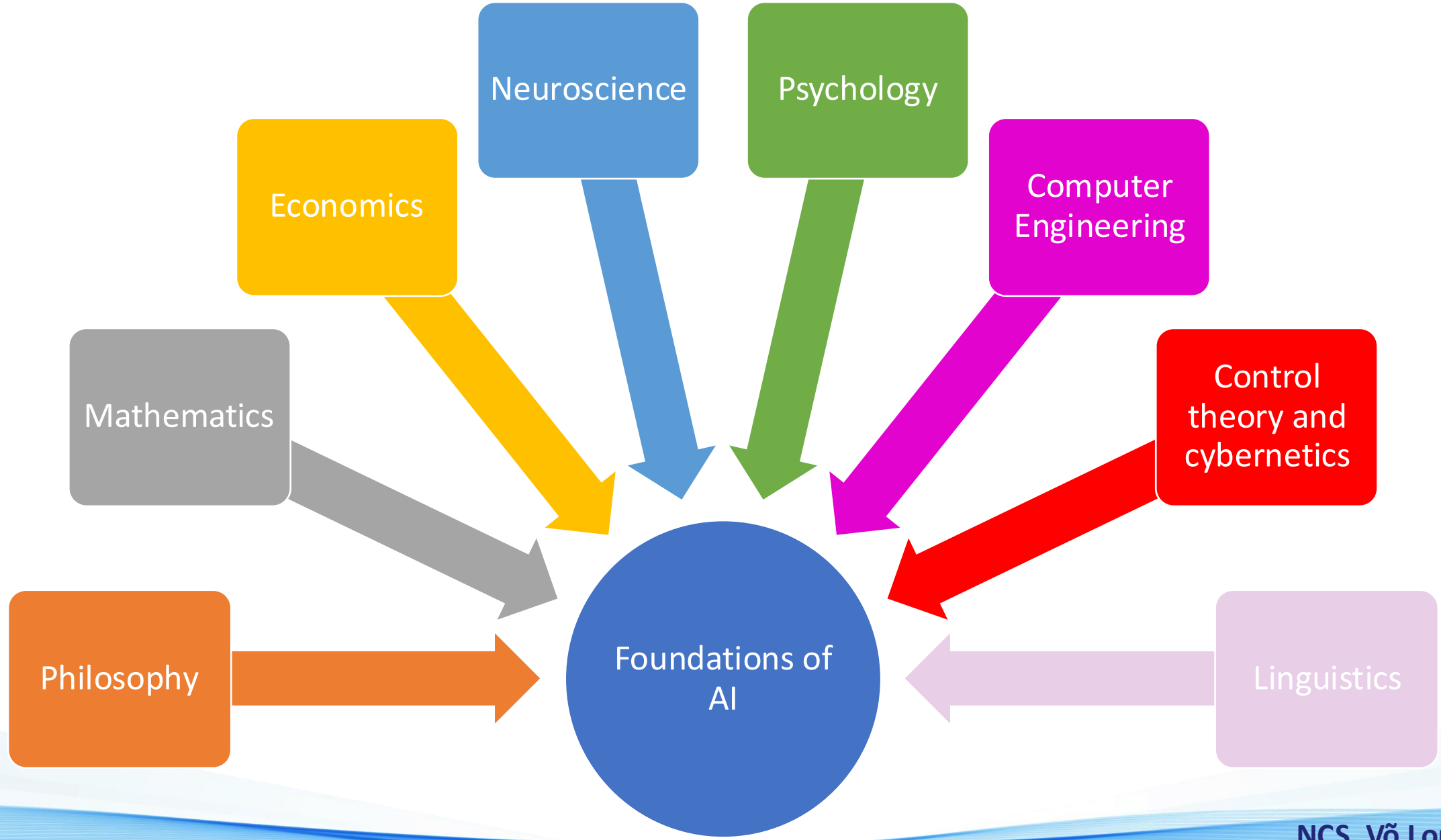
Acting humanly: The Turing test approach

NLP – knowledge representation – Automated reasoning - ML

Acting rationally: The rational agent approach

AI Agents

# Foundations of AI



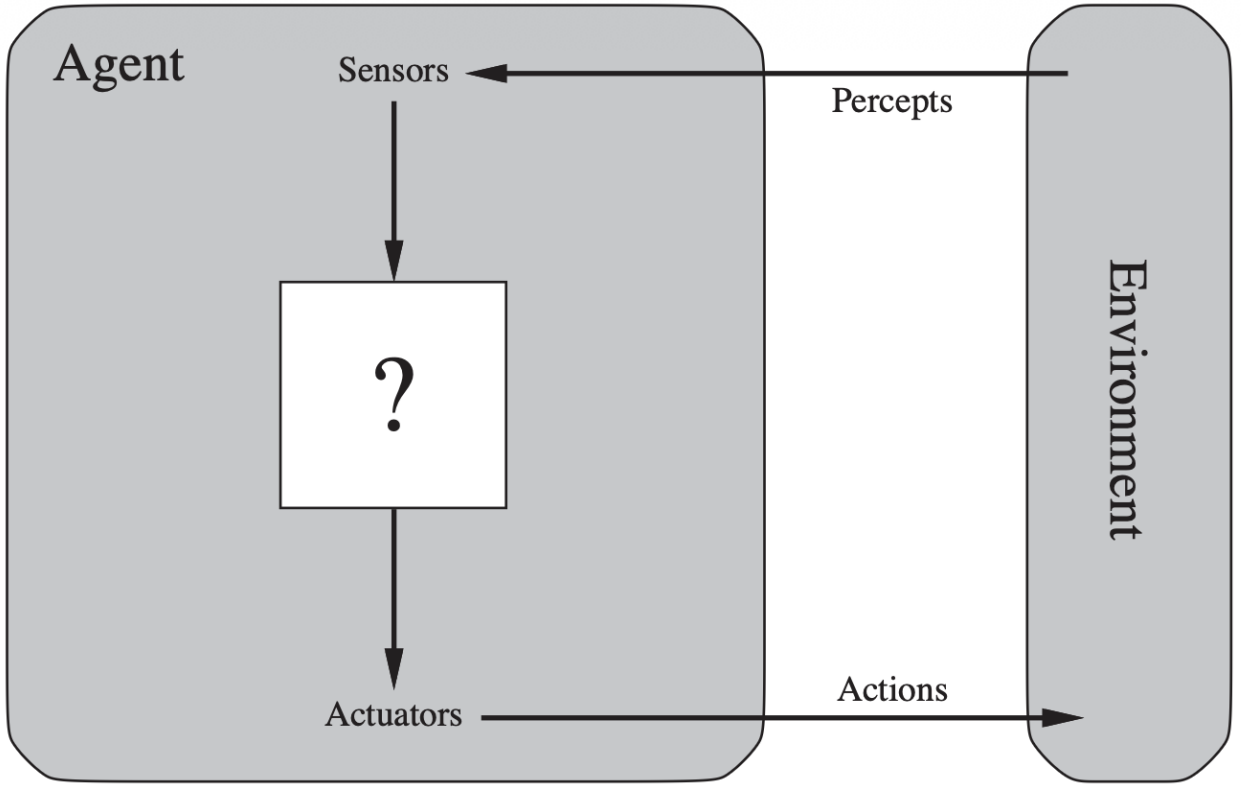
# Foundations of AI

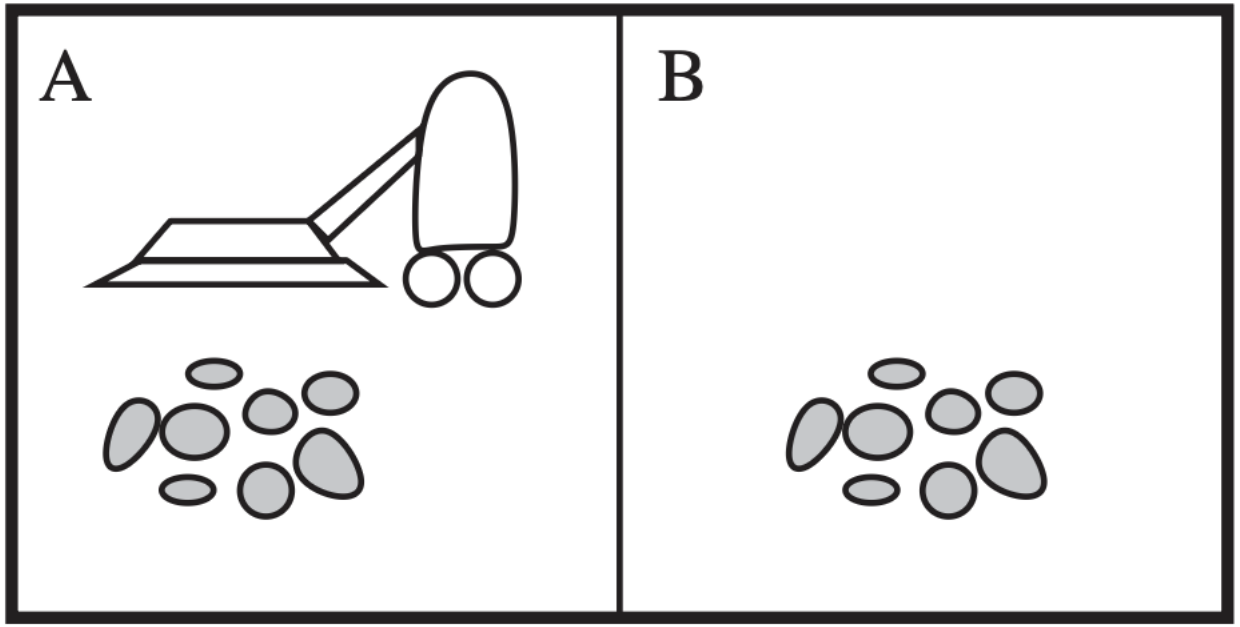
| Discipline | Contribution to AI |
|---|---|
| Philosophy | Logic, methods of reasoning, mind as physical system, foundations of learning, language, rationality |
| Mathematics | Formal representation and proof algorithms, computation, (un)decidability, (in)tractability, probability |
| Economics | Utility, decision theory |
| Neuroscience | Physical substrate for mental activity |
| Psychology | Phenomena of perception and motor control, experimental techniques |
| Computer engineering | Building fast computers |
| Control theory | Design systems that maximize an objective function over time |
| Linguistics | Knowledge representation, grammar |

- An **agent** is anything that can be considered as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.

  Ex: a human agent, a robotic agent, …
- A **percept** refers to the agent's perceptual inputs, while an agent's **percept sequence** is the complete history of everything the agent has ever perceived.
- An agent's behavior:

  **agent function(percept sequence) = action**






A vacuum-cleaner world with just two locations.

**Location** = {A, B}

**State** = {Clean, Dirty}

**Percept** ← Environment(location; state)

**Actions** = {Left, Right, Suck}

Percept sequence (per.seq) could be

- Per.seq 1 = (A, Clean)
- Per.seq 2 = (A, Clean), (A Dirty)
- Per.seq 3 = (A, Clean), (B Dirty), …

Agent function (Per.seq 1) = Agent function (A, Clean) = Right

Agent function (Per.seq 2) = Agent function ((A, Clean), (A Dirty)) = Suck

We can imagine tabulating the agent function that describes any given agent; for most agents, this would be a very large table - infinite, in fact, unless we place a bound on the length of percept sequences.

Q: What is the right way to fill out the table? In other words, what makes an agent good or bad, intelligent or stupid?

| Percept sequence | Action |
|---|---|
| $[A, Clean]$ | $Right$ |
| $[A, Dirty]$ | $Suck$ |
| $[B, Clean]$ | $Left$ |
| $[B, Dirty]$ | $Suck$ |
| $[A, Clean], [A, Clean]$ | $Right$ |
| $[A, Clean], [A, Dirty]$ | $Suck$ |
| ⋮ | ⋮ |
| $[A, Clean], [A, Clean], [A, Clean]$ | $Right$ |
| $[A, Clean], [A, Clean], [A, Dirty]$ | $Suck$ |
| ⋮ | ⋮ |

Partial tabulation of a simple agent function for the vacuum-cleaner world

# 2.2. Rational agents

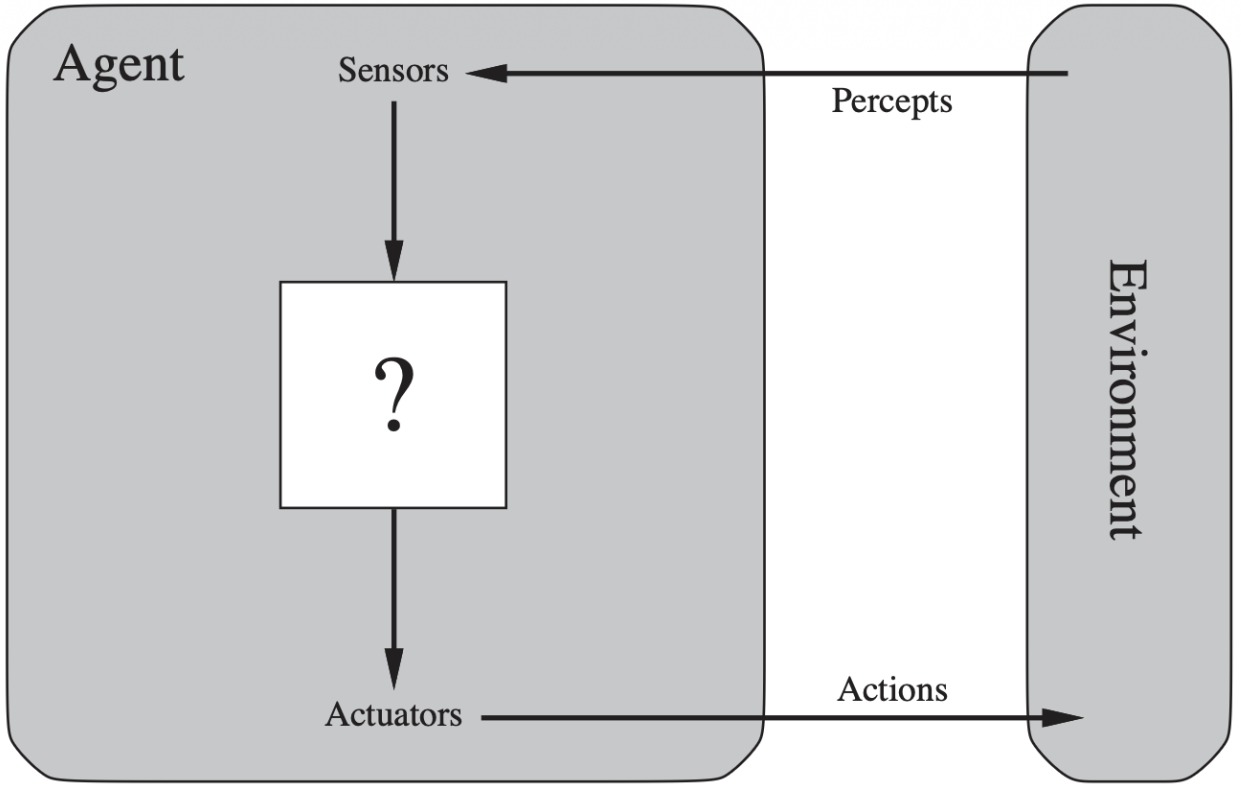
- A rational agent is one that does the right thing (table for the agent function is filled out correctly).

→ What does it mean to do the right thing?

- **Performance measure** evaluates any given sequence of **environment states.**

Example:

- The amount of dirt cleaned up
- The amount of electricity consumed
- The amount of noise generated
- The amount of time taken

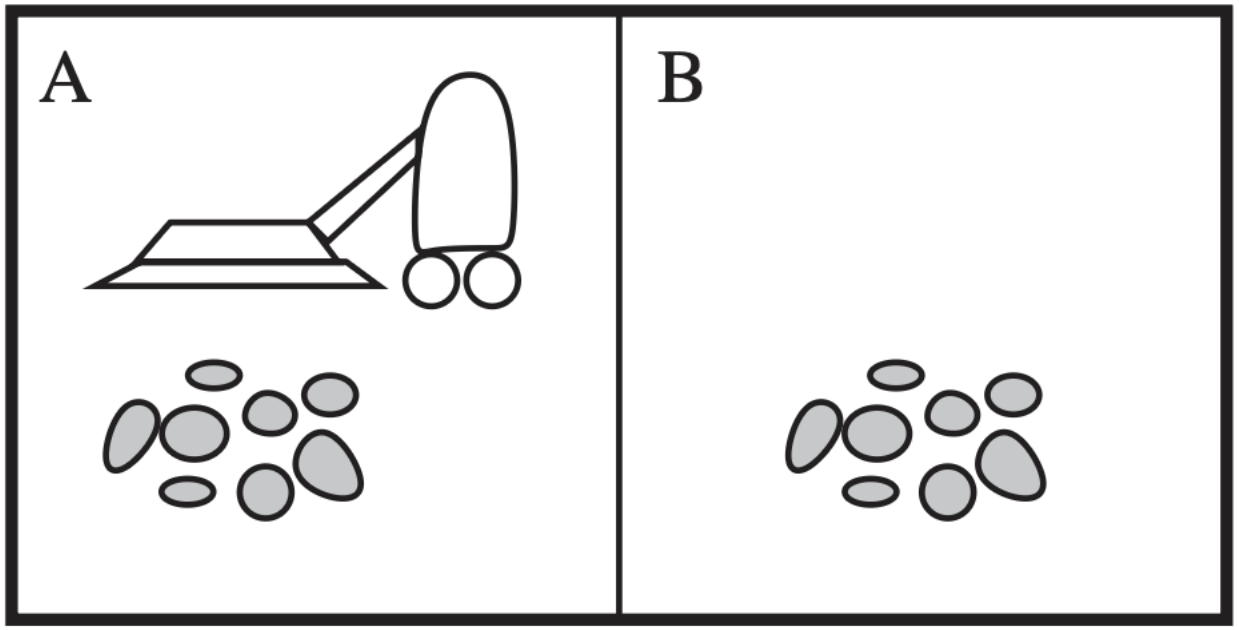


# 2.2. Rational agents

*A **rational agent** should select an **action** that is expected to maximize its **performance measure**, based on its **percept sequence** and its built-in **knowledge.***

- Ations: Left, Right, and Suck.
- Performance measure: awards 1 point for each clean square at each time step, a limit of 1000 time steps.
- The environment is known: location/state
- The agent correctly perceives its location and whether that location contains dirt.

→ Under *these circumstances*, the agent is indeed rational.




A vacuum-cleaner world with just two locations.

# 2.3. The nature of environments: PEAS

To design an automated agent, we should consider the **PEAS** description**:**

**P**erformance, **E**nvironment, **A**ctuators, **S**ensors

| Agent Type | Performance Measure | Environment | Actuators | Sensors |
|---|---|---|---|---|
| Taxi driver | Safe, fast, legal, comfortable trip, maximize profits | Roads, other traffic, pedestrians, customers | Steering, accelerator, brake, signal, horn, display | Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard |

PEAS description of the task environment for an automated taxi.

**TASK:** Sketch the PEAS elements for tow agent types: medical diagnosis system and interactive English tutor

# 2.3. The nature of environments: PEAS

**TASK:** Sketch the PEAS elements for tow agent types: medical diagnosis system and interactive English tutor

| Agent Type | Performance Measure | Environment | Actuators | Sensors |
|---|---|---|---|---|
| Medical diagnosis system | Healthy patient, reduced costs | Patient, hospital, staff | Display of questions, tests, diagnoses, treatments, referrals | Keyboard entry of symptoms, findings, patient's answers |
| Interactive English tutor | Student's score on test | Set of students, testing agency | Display of exercises, suggestions, corrections | Keyboard entry |

# Properties of task environments

FULLY OBSERVABLE

Fully observable



PARTIALLY OBSERVABLE

Partially observable

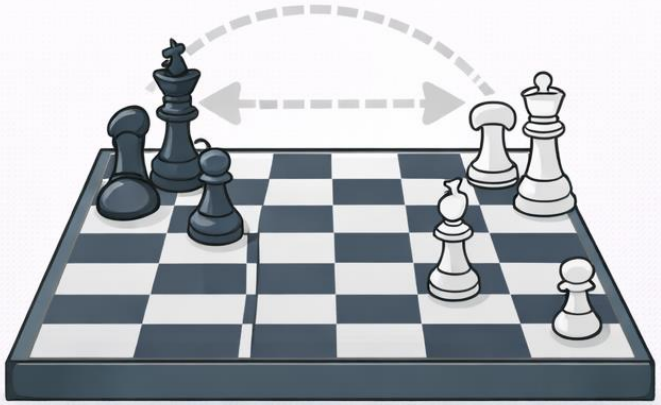
**Property 1**

- **Fully observable**: The agent can see all the relevant aspects of the environment needed to choose the best action.
- **Partially observable**: The agent can see only part of the environment, so it must deal with missing or uncertain information.

# Properties of task environments

**SINGLE AGENT**

One agent acting alone



**MULTIAGENT**

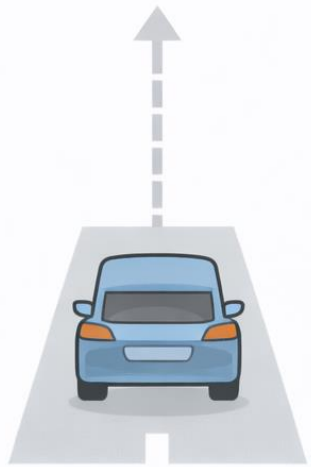
Multiple agents acting together

- Competitive or
- Cooperative

**Property 2**

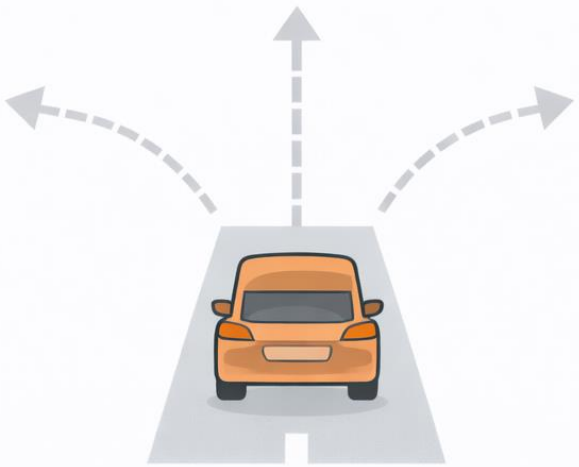
- **Single-agent:** Only one agent is acting, and its performance does not depend on other agents.
- **Multiagent:** Multiple agents act in the environment, and their actions affect each other's performance.

# Properties of task environments



DETERMINISTIC

Future outcome is predictable



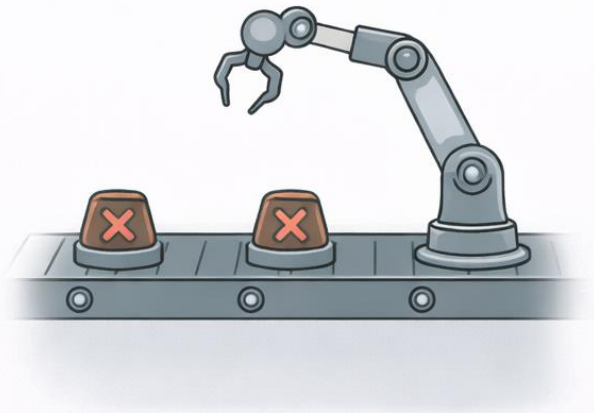
STOCHASTIC

Future outcome is uncertain

**Property 3**

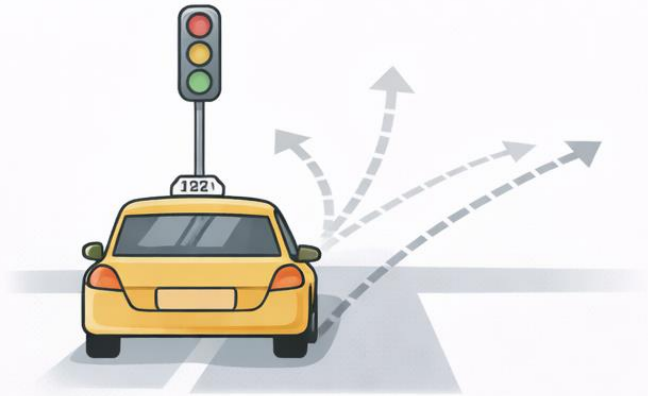
- **Deterministic:** The next state of the environment is completely determined by the current state and the agent's action.
- **Stochastic:** The next state of the environment involves randomness, so the same action can lead to different outcomes.

# Properties of task environments



EPISODIC

Episodes are independent



SEQUENTIAL

Episodes depend on previous ones

**Property 4**

- **Episodic:** Each decision is independent, and current actions do not affect future episodes.
- **Sequential:** Current actions influence future states and future decisions.

# Properties of task environments

STATIC

Environment stays the same

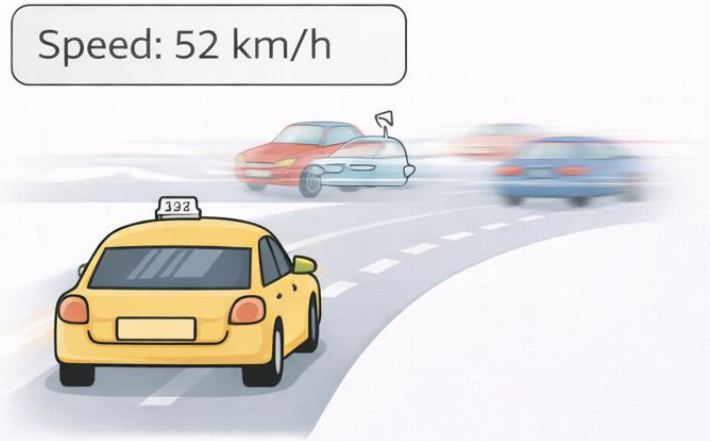
DYNAMIC

Enivironment changes over time

**Property 5**

- **Static:** The environment does not change while the agent is thinking.
- **Dynamic:** The environment can change while the agent is deliberating.
- **Semidynamic:** The environment does not change but the agent's performance score does.

# Properties of task environments




**DISCRETE**

Limited distinct states a d actions

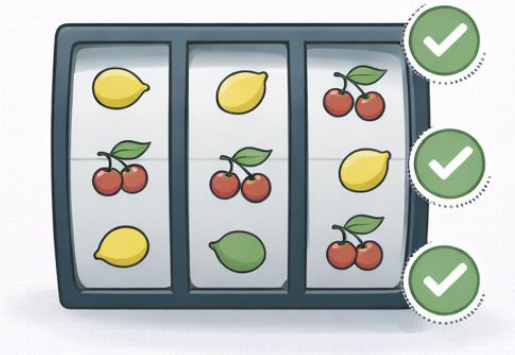
**CONTINUOUS**

Infinitely varying states and actions

**Property 6**

- **Discrete:** The environment has a finite or countable number of distinct states, percepts, and actions.
- **Continuous**: The environment involves continuously varying states, time, percepts, or actions.

# Properties of task environments



KNOWN

The agent knows the outcomes

UNKNOWN

The agent is uncertain about the outcomes

**Property 7**

- **Known:** The agent knows how the environment works, including the outcomes (or probabilities) of its actions.
- **Unknown:** The agent does not know the rules of the environment and must learn them through experience.

# Properties of task environments

| Task Environment | Observable | Agents | Deterministic | Episodic | Static | Discrete |
|---|---|---|---|---|---|---|
| Crossword puzzle | | | | | | |
| Chess with a clock | | | | | | |
| Poker | | | | | | |
| Taxi driving | | | | | | |
| Medical diagnosis | | | | | | |
| Image analysis | | | | | | |
| Interactive English tutor | | | | | | |

# Properties of task environments

| Task Environment | Observable | Agents | Deterministic | Episodic | Static | Discrete |
|---|---|---|---|---|---|---|
| Crossword puzzle | Fully | Single | Deterministic | Sequential | Static | Discrete |
| Chess with a clock | Fully | Multi | Deterministic | Sequential | Semi | Discrete |
| Poker | Partially | Multi | Stochastic | Sequential | Static | Discrete |
| Taxi driving | Partially | Multi | Stochastic | Sequential | Dynamic | Continuous |
| Medical diagnosis | Partially | Single | Stochastic | Sequential | Dynamic | Continuous |
| Image analysis | Fully | Single | Deterministic | Episodic | Semi | Continuous |
| Interactive English tutor | Partially | Multi | Stochastic | Sequential | Dynamic | Discrete |

# 2.4. The structure of agents

Recall: An agent's behavior:

**agent function(percept sequence ) = action**

- *An **agent program** is an implementation of agent function.*

We assume this program will run on some sort of computing device with physical sensors and actuators; and we call this the ***architecture***.

# 2.4.1. Agent programs

An **agent program** is an implementation of agent function. However, it takes just the current percept as input because nothing more is available from the environment; if the agent's actions need to depend on the entire percept sequence, the agent will have to remember the percepts.

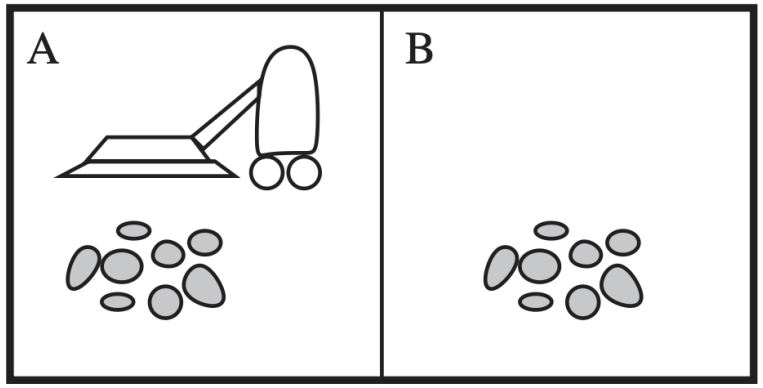
There is an easy way to implement agent programs, which is to use a table-driven approach. Unfortunately, that seems impossible in practice.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
              table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

# 2.4.1. Agent programs




Let P be the set of possible percepts and let T be the lifetime of the agent. The number of entries in the lookup table will be

$$\sum_{t=1}^{T} |P|^t$$

| Percept sequence | Action |
|---|---|
| $[A, Clean]$ | $Right$ |
| $[A, Dirty]$ | $Suck$ |
| $[B, Clean]$ | $Left$ |
| $[B, Dirty]$ | $Suck$ |
| $[A, Clean], [A, Clean]$ | $Right$ |
| $[A, Clean], [A, Dirty]$ | $Suck$ |
| ⋮ | ⋮ |
| $[A, Clean], [A, Clean], [A, Clean]$ | $Right$ |
| $[A, Clean], [A, Clean], [A, Dirty]$ | $Suck$ |
| ⋮ | ⋮ |

$4$

$4^2$

$4^3$

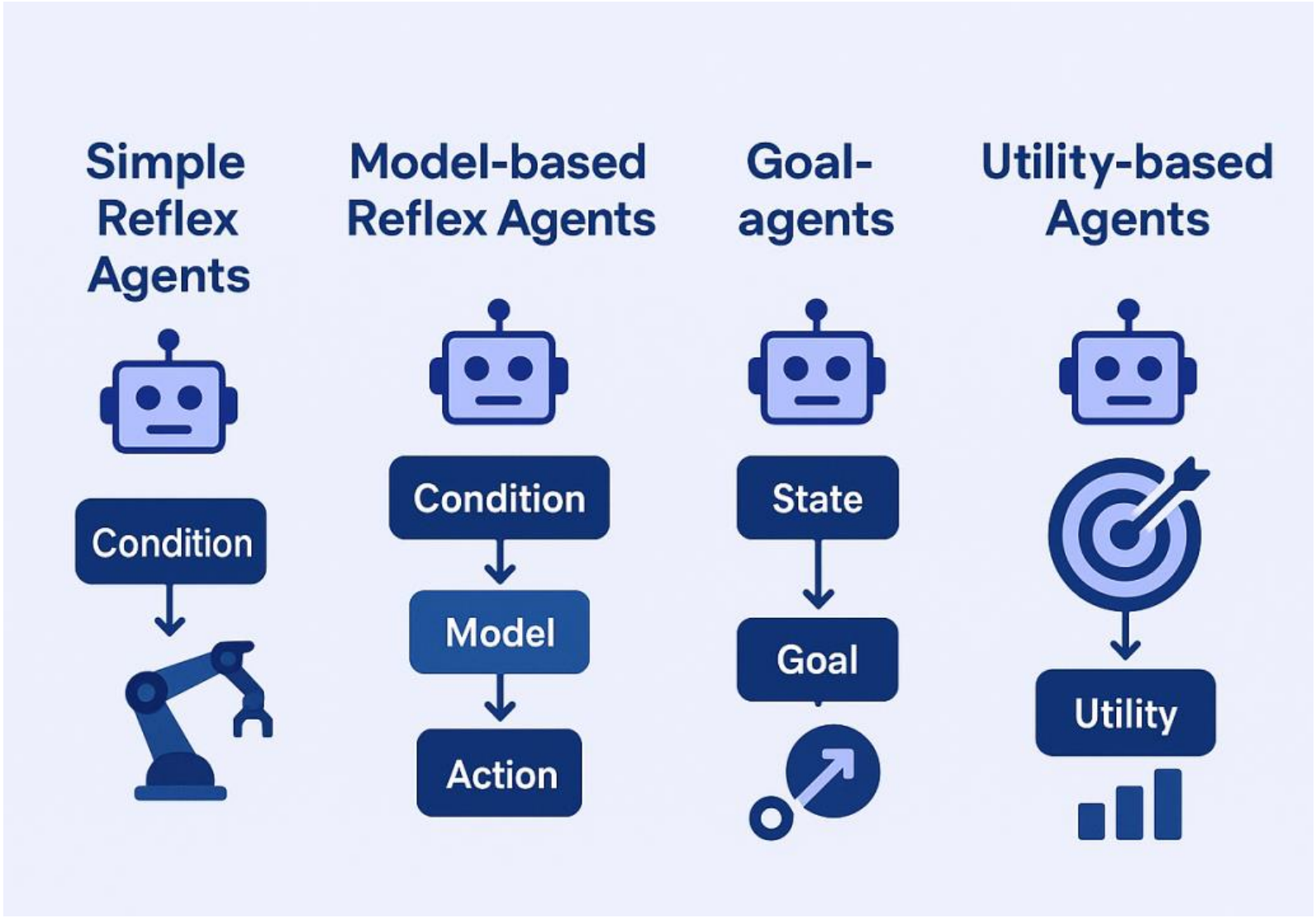
1 action / 10 seconds

1 hour = 360

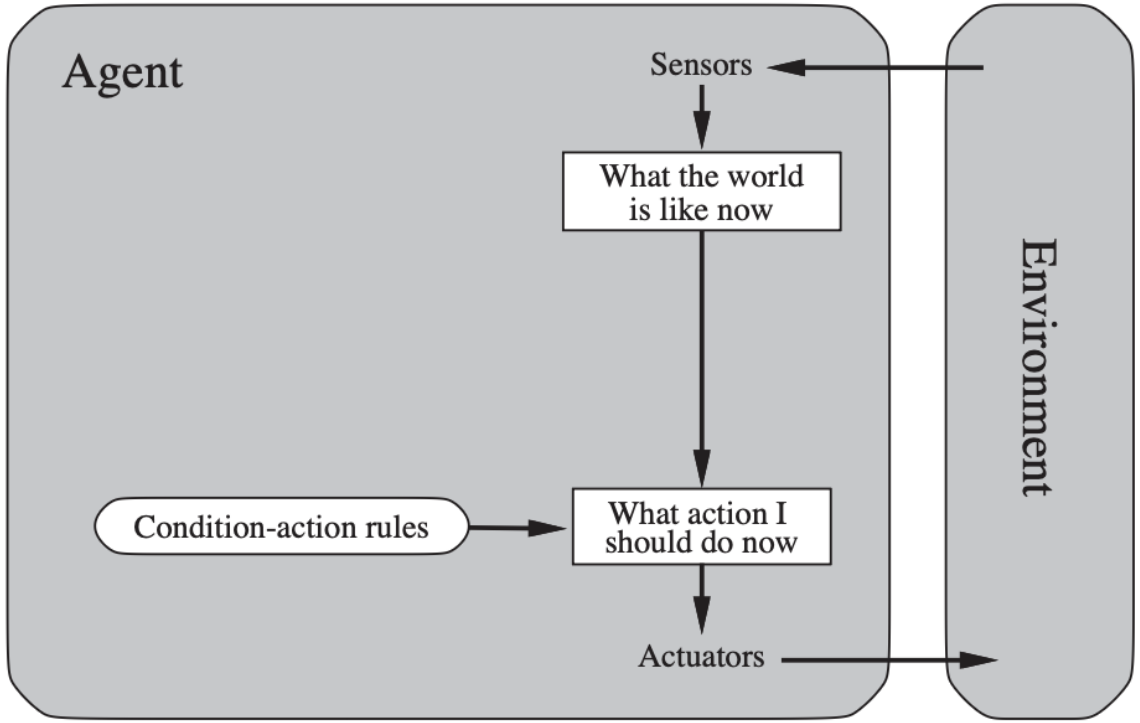
That number will be

$$4^1 + 4^2 + \cdots + 4^{360} = \frac{4}{3}(4^{360} - 1)$$

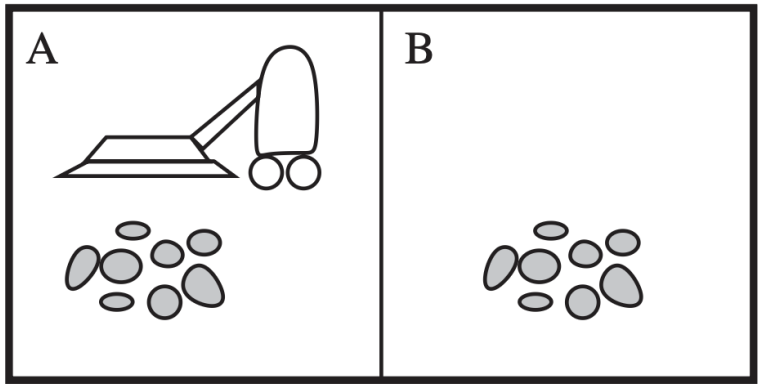
# 2.4.2. Agent types



# Simple reflex agents




**A simple reflex agent** is the simplest kind of agent, this selects actions on the basis of the current percept, ignoring the rest of the percept history.



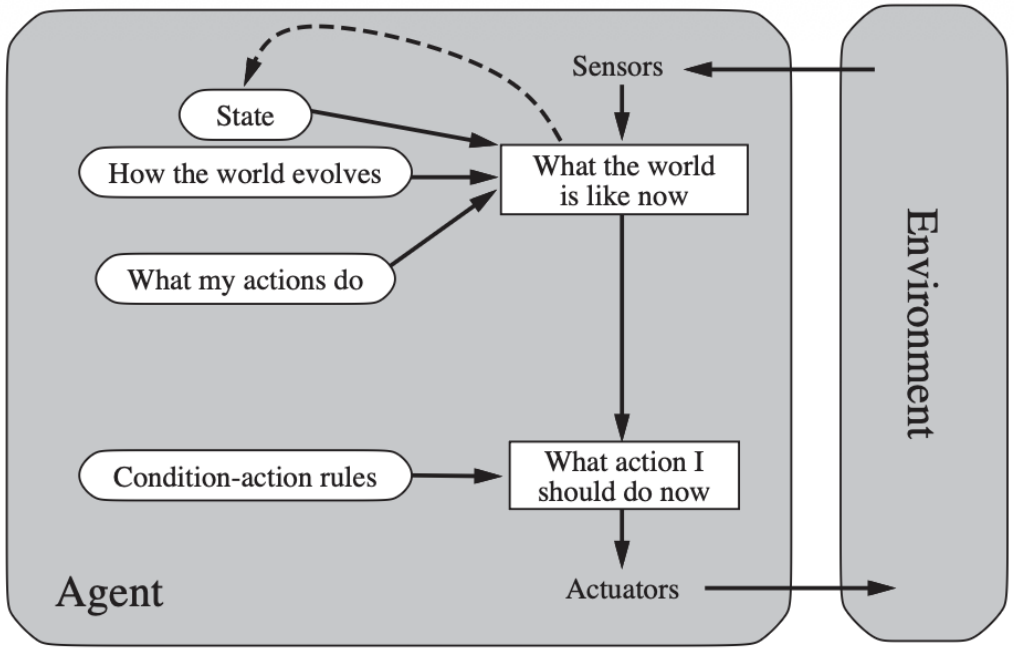

**function** SIMPLE-REFLEX-AGENT(*percept*) **returns** an action
  **persistent**: *rules*, a set of condition–action rules

  *state* ← INTERPRET-INPUT(*percept*)
  *rule* ← RULE-MATCH(*state*, *rules*)
  *action* ← *rule*.ACTION
  **return** *action*

**function** REFLEX-VACUUM-AGENT([*location*,*status*]) **returns** an action

  **if** *status* = *Dirty* **then return** *Suck*
  **else if** *location* = *A* **then return** *Right*
  **else if** *location* = *B* **then return** *Left*

# Model-based reflex agents



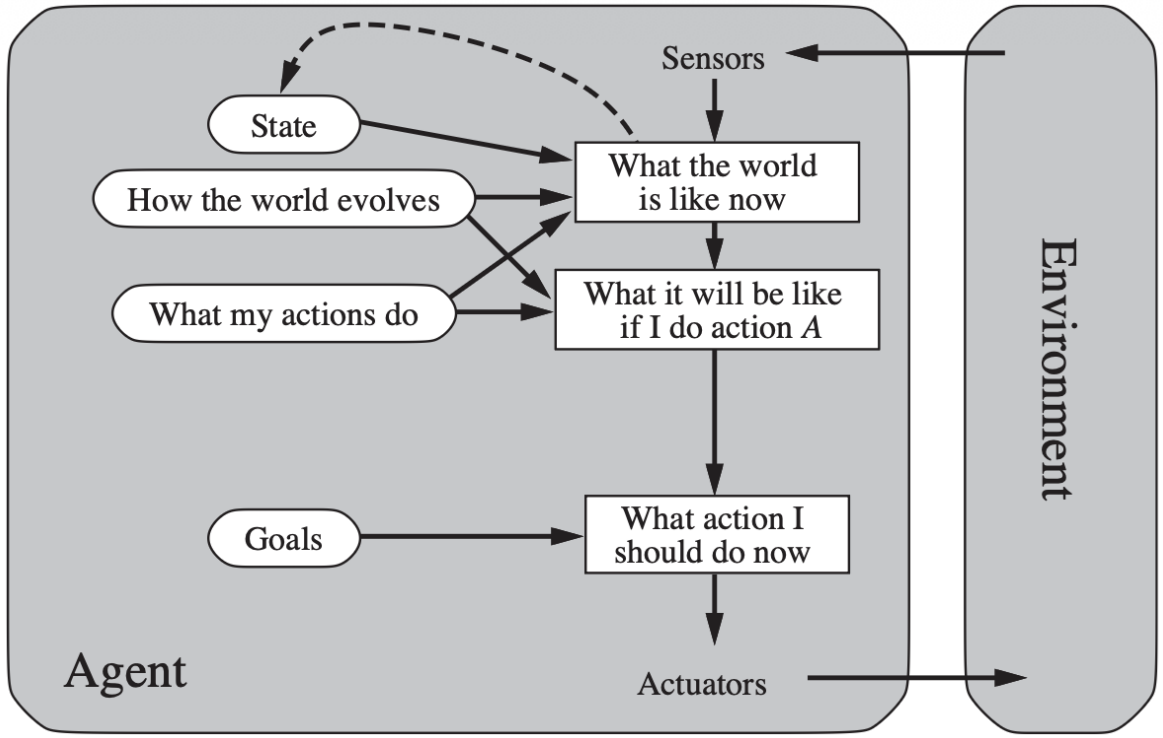

**A model-based agent** is an agent that has ***internal state*** and a ***model of the world*** to handle a partially observable environment.

- ***Internal state:*** The best guess current world, including unobservable things.
- ***Model:*** How the world works.

**function** MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action
  **persistent**: *state*, the agent's current conception of the world state
    *model*, a description of how the next state depends on current state and action
    *rules*, a set of condition–action rules
    *action*, the most recent action, initially none

  *state* ← UPDATE-STATE(*state*, *action*, *percept*, *model*)
  *rule* ← RULE-MATCH(*state*, *rules*)
  *action* ← *rule*.ACTION
  **return** *action*

# Gold-based agents



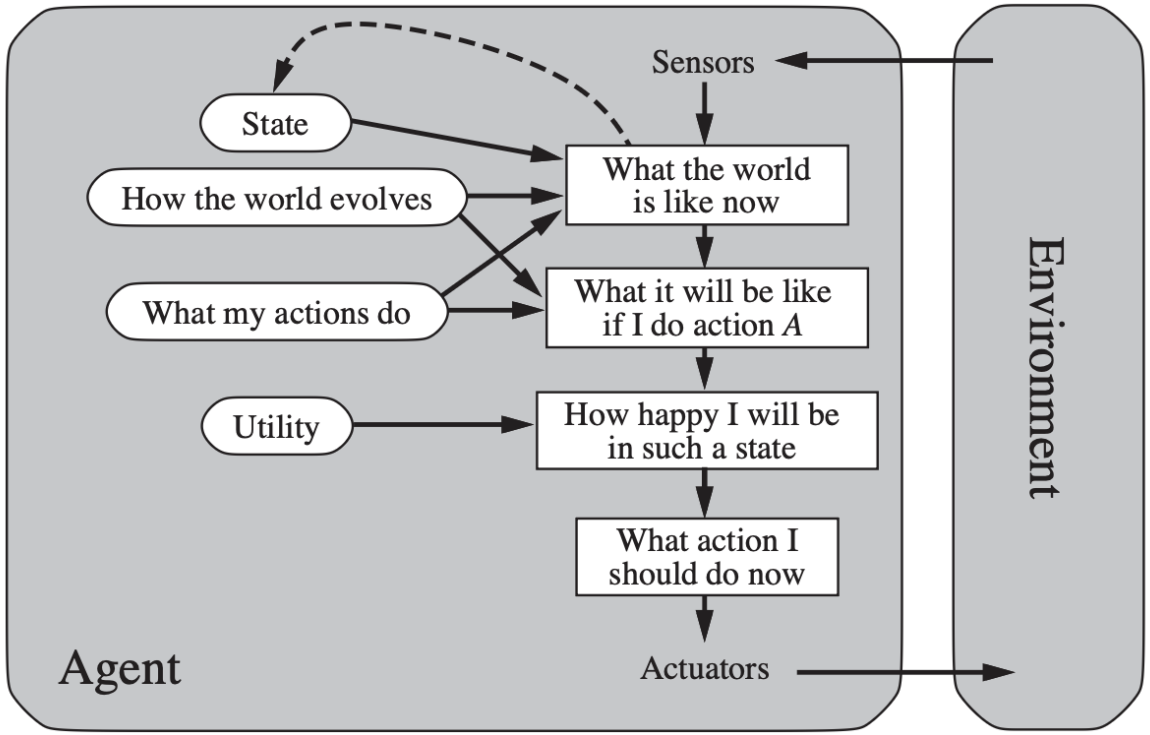

A model-based, goal-based agent.

Knowing something about the current state of the environment is not always enough to decide what to do. For example, at a road junction, the taxi can turn left, turn right, or go straight on. The correct decision depends on where the taxi is trying to get to.

**A gold-based agent** is an agent that makes decisions based on goals and future predictions.

- Model
- Gold
- Search/planning

# Utility-based agents




A model-based, utility-based agent.

For gold-based agents:

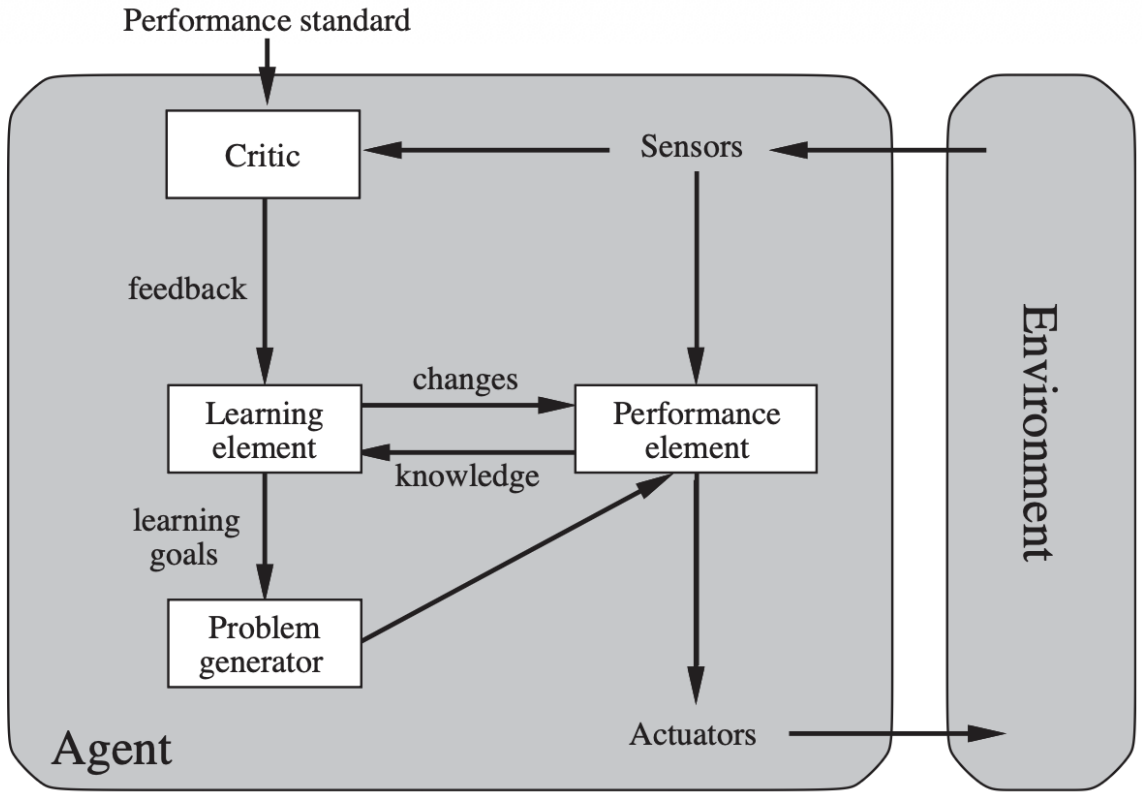
- Achieve gold → good
- Do not achieve gold → bad

For example, in a taxi driver agent, there are many ways to get the gold destination.

**A utility-based agent** acts to maximize expected utility rather than merely achieving a goal.

**Utility function:** U(state) $\in (0; 100)$

# Learning agents




- **Performance element:** selects actions based on percepts.
- **Learning element:** makes improvements to the performance element.
- **Critic:** evaluates performance using a fixed performance standard and provides feedback. The performance standard must remain fixed and should not be modified by the agent.
- **Problem generator:** suggests exploratory actions to gain useful experience.


Trung tâm Học liệu và Dạy học số

Đại học Công nghệ Kỹ thuật Tp. Hồ Chí Minh

01 Võ Văn Ngân, P. Thủ Đức, Tp.Hồ Chí Minh

**daotaotuxa@hcmute.edu.vn**